

ElevatorSimulation



- Simulation d'un ascenseur
 - Auteur : ELKHALKI Yassine
 - Contact :
 - yassine.elkhalki@outlook.fr
 - yassine.elkhalki.auditeur@lecnam.net
-

Sommaire

1. [Consignes](#)
2. [Implémentation](#)
3. [Architecture technique](#)
 - [Code source](#)
 - [Bibliothèques](#)
 - [Déroulement](#)
 - [Affichage](#)
4. [Usage](#)
5. [Tests](#)

Consignes : Simulation d'un ascenseur (threads et mutex)

Contexte :

- Implémentez un système d'ascenseur pour un bâtiment de 5 étages.
- Les utilisateurs (threads) demandent à monter ou descendre.

Fonctionnalités à implémenter :

- Gestion des requêtes d'appel et de destination.
- Simulation du mouvement de l'ascenseur.
- Synchronisation des utilisateurs pour accéder à l'ascenseur.

Objectifs pédagogiques :

- Synchronisation avancée des threads.
- Gestion des événements concurrents.

Implémentation

- L'ascenseur implémenté est un petit ascenseur. Il ne peut prendre qu'une personne à la fois.
- Une V2 est en cours d'implémentation (branche **v2**) qui permet de prendre plusieurs passagers se basant sur l'algorithme SCAN (Elevator Algorithm).

Cette version repose sur le modèle Producteur-Consommateur décrit dans le cours. J'utilise donc :

- Deux mutex (***pthread_mutex_t***). Un pour garantir l'exclusion mutuelle lors de l'accès au buffer de requête partagé. Un autre pour s'assurer que les requêtes de tous les passagers ont été satisfaites.
- Deux sémaphores (***sem_t***).
 - **empty** : pour dire si l'ascenseur est libre ou non
 - **full** : pour dire qu'un passager est prêt à être transporté

La simulation propose un affichage pas à pas dans le terminal pour suivre l'ascenseur à travers les 5 étages. Voici un aperçu du rendu :

```
=== Lancement simulation Ascenseur avec 5 usagers ===
```

```

RDC | 1 | 2 | 3 | 4 | ETAT ASCENSEUR
-----
[ E ] | | | | | <-- En attente...
[PASSAGER 0] Je veux aller de 0 à 4.
[ASCENSEUR] PRISE EN CHARGE PASSAGER 0 !
[ E ] | | | | | <-- Ouverture des portes ...
[PASSAGER 1] Je veux aller de 0 à 4.
[PASSAGER 2] Je veux aller de 0 à 1.
[PASSAGER 3] Je veux aller de 0 à 3.
[PASSAGER 4] Je veux aller de 2 à 1.
[ P0 ] | | | | | <-- Passager P0 est monté
[ P0 ] | | | | | <-- Fermeture des portes ...
| [ P0 ] | | | | <-- Transport P0 -> 4
| | [ P0 ] | | | <-- Transport P0 -> 4
| | | [ P0 ] | | <-- Transport P0 -> 4
| | | | [ P0 ] | <-- Transport P0 -> 4
| | | | [ P0 ] <-- Ouverture des portes ...
| | | | [ E ] <-- Passager P0 est descendu
[ASCENSEUR] PRISE EN CHARGE PASSAGER 1 !
| | | | [ E ] | <-- Je vais chercher P1 à l'étage 0
| | | [ E ] | | <-- Je vais chercher P1 à l'étage 0
| [ E ] | | | | <-- Je vais chercher P1 à l'étage 0
[ E ] | | | | | <-- Je vais chercher P1 à l'étage 0
[ E ] | | | | | <-- Ouverture des portes ...
[ P1 ] | | | | | <-- Passager P1 est monté
[ P1 ] | | | | | <-- Fermeture des portes ...
| [ P1 ] | | | | <-- Transport P1 -> 4
| | [ P1 ] | | | <-- Transport P1 -> 4
| | | [ P1 ] | | <-- Transport P1 -> 4
| | | | [ P1 ] | <-- Transport P1 -> 4
| | | | [ P1 ] <-- Ouverture des portes ...
| | | | [ E ] <-- Passager P1 est descendu
[ASCENSEUR] PRISE EN CHARGE PASSAGER 2 !
| | | | [ E ] | <-- Je vais chercher P2 à l'étage 0
| | | [ E ] | | <-- Je vais chercher P2 à l'étage 0
| [ E ] | | | | <-- Je vais chercher P2 à l'étage 0
[ E ] | | | | | <-- Je vais chercher P2 à l'étage 0
[ E ] | | | | | <-- Ouverture des portes ...
[ P2 ] | | | | | <-- Passager P2 est monté
[ P2 ] | | | | | <-- Fermeture des portes ...

```

```

| [ P2] | | | | <-- Transport P2 -> 1
| [ P2] | | | | <-- Ouverture des portes ...
| [ E ] | | | | <-- Passager P2 est descendu
[ASCENSEUR] PRISE EN CHARGE PASSAGER 3 !
[ E ] | | | | <-- Je vais chercher P3 à l'étage 0
[ E ] | | | | <-- Ouverture des portes ...
[ P3] | | | | <-- Passager P3 est monté
[ P3] | | | | <-- Fermeture des portes ...
| [ P3] | | | | <-- Transport P3 -> 3
| | [ P3] | | | | <-- Transport P3 -> 3
| | | [ P3] | | | | <-- Transport P3 -> 3
| | | [ P3] | | | | <-- Ouverture des portes ...
| | | [ E ] | | | | <-- Passager P3 est descendu
[ASCENSEUR] PRISE EN CHARGE PASSAGER 4 !
| | [ E ] | | | | <-- Je vais chercher P4 à l'étage 2
| | [ E ] | | | | <-- Ouverture des portes ...
| | [ P4] | | | | <-- Passager P4 est monté
| | [ P4] | | | | <-- Fermeture des portes ...
| [ P4] | | | | <-- Transport P4 -> 1
| [ P4] | | | | <-- Ouverture des portes ...
| [ E ] | | | | <-- Passager P4 est descendu

```

=== Fin simulation ===

Architecture technique

Code source

Ce dépôt est décomposé en deux sous-dossiers :

- **doc** : Qui contient la documentation (le sujet ainsi que le [compte rendu](#))
- **src** : Qui contient le code source du projet dont :
 - **simulation.h** : Fichier qui contient la définition des prototypes et de la structure Passager
 - **simulation_v1.c** : Fichier qui contient toute la logique métier de la version 1 notamment les fonctions des threads (ascenseur/usager), la gestion des sémaphores/mutex et la fonction d'affichage graphique
 - **simulation_v2.c** : Fichier de la version 2 en cours d'implémentation.
 - **main.c** : Point d'entrée du programme qui gère la récupération des arguments (le nombre d'usager) et lance la simulation

Bibliothèques

La projet repose sur des bibliothèques standards du langage C et des bibliothèques POSIX pour la gestion des threads:

- **<pthread.h>** : Fournit les fonctions nécessaires à la création et à la gestion des threads ainsi que les mutex
- **<semaphore.h>** : Utilisée pour contrôler l'accès aux ressources partagées et synchroniser les flux producteurs-consommateurs

- **<fnctl.h>** : Car développement sur macOS pour l'utilisation des constantes lors de l'initialisation des sémaphores nommés
- **<unistd.h>** : permet l'utilisation des fonctions sleep et usleep pour simuler les temps de trajets

Déroulement

- **Fonction run_simulation_v1** : Il s'agit de la fonction qui initialise mes variables globales ainsi que les passagers de manière aléatoire dans une boucle de sorte à ce qu'il n'y ait pas de passagers qui ont le même étage de départ que d'arrivée. Je n'ai mis aucune de simulation de temps d'attente entre la création de chaque passager car je voulais éviter que les passager s'approprient l'ascenseur dans l'ordre et qu'il n'y ait aucune concurrence visible.
- **Fonction thread_usager** : Il s'agit de la fonction thread qui pour chaque passager va vérifier que l'ascenseur est libre puis avec un mutex s'approprier l'ascenseur afin d'éviter que deux passager s'approprient l'ascenseur en même temps.
- **Fonction thread_ascenseur** : Il s'agit de la fonction qui attend les passager et simule le déroulement de l'ascenseur. Il y a des sleep pour simuler les pauses et les temps d'attentes.
- **Fonction afficher_etat** : Il s'agit d'une fonction d'affichage pour gérer l'affichage un peu à la manière d'un diagramme d'état de l'ascenseur et des passagers.

Affichage

L'affichage est géré sous forme de diagramme d'états avec les étages des l'ascenseur et l'état de l'ascenseur. Exemples avec l'affichage précédent.

- Les requêtes des passagers sont affichées en lignes et le signalement de prise en charge de la part de l'ascenseur également
 - **Requête** : [PASSAGER 0] Je veux aller de 0 à 4.
 - **Signalement de prise en charge** : [ASCENSEUR] PRISE EN CHARGE PASSAGER 1 !
- Les déplacements de l'ascenseurs sont affichés dans les colonnes des étages
 - **[E]** : Pour ascenseur vide en déplacement ou en attente
 - **[Pn]** : Pour ascenseur avec le passager n
- L'état de l'ascenseur est décrit dans la colonne ETAT ascenseur

Usage

Tout d'abord, clonez le projet :

```
git clone https://github.com/YassouSensai/ElevatorSimulation.git
```

Pour compiler le projet, il faut avoir le compilateur **gcc** d'installé sur votre machine. Pour compiler et obtenir l'exécutable sur Linux et macOS, rendez vous à la racine du projet, c'est à dire [ici](#) puis exécutez la commande suivante :

```
gcc -Wall -pthread src/main.c src/simulation_v1.c -o ElevatorSimulation
```

un exécutable du nom de **ElevatorSimulation** sera créé et il attend en paramètre le nom de la version, le nombre d'utilisateurs et pour la version 2 le nombre de passagers maximal.

Pour exécuter la v1 :

```
./ElevatorSimulation v1 4
```

Pour exécuter la v2 :

```
./ElevatorSimulation v1 4
```

Finalement pour supprimer l'exécutable :

```
rm ElevatorSimulation
```

Tests

Les tests d'affichages sont fait sur la base de l'exécution suivante :

```
./ElevatorSimulation v1 10
```

- Ascenseur qui monte puis descend & passagers pris dans le mauvais ordre de la boucle de créations des passagers :
 - Le passager 3 est pris en charge au RDC (étage 0), puis l'ascenseur se rend à l'étage de destination qui est le 4. Puis l'ascenseur prend en charge le passager 1 à l'étage 3 donc il descend le chercher et le ramène à destination au deuxième en descendant.
 - Le passager 3 est pris en charge avant le passager 1 => le thread du p3 est arrivé avant celui du P1 à l'ascenseur

```
[ASCENSEUR] PRISE EN CHARGE PASSAGER 3 !
[ E ] |   |   |   |   |   <-- Je vais chercher P3 à l'étage 0
[ E ] |   |   |   |   |   <-- Ouverture des portes ...
[ P3] |   |   |   |   |   <-- Passager P3 est monté
[ P3] |   |   |   |   |   <-- Fermeture des portes ...
    | [ P3] |   |   |   |   <-- Transport P3 -> 4
    |   | [ P3] |   |   |   <-- Transport P3 -> 4
    |   |   | [ P3] |   |   <-- Transport P3 -> 4
    |   |   |   | [ P3] |   <-- Transport P3 -> 4
    |   |   |   | [ P3] |   <-- Ouverture des portes ...
    |   |   |   | [ E ] |   <-- Passager P3 est descendu
[ASCENSEUR] PRISE EN CHARGE PASSAGER 1 !
```

```
| | | [ E ] | <-- Je vais chercher P1 à l'étage 3
| | | [ E ] | <-- Ouverture des portes ...
| | | [ P1] | <-- Passager P1 est monté
| | | [ P1] | <-- Fermeture des portes ...
| | [ P1] | | <-- Transport P1 -> 2
| | [ P1] | | <-- Ouverture des portes ...
| | [ E ] | | <-- Passager P1 est descendu
```